

Food Delivery Order Management

Food Delivery Order Management

Time limit: 48 hours from assignment. **Stack:** Spring Boot.

The Project

A food delivery order management system at scale with multiple restaurants across cities, menu management per restaurant, customer order placement, the order lifecycle (placed → accepted → preparing → out-for-delivery → delivered), and delivery-partner assignment. Concurrent orders for the same menu item should not oversell limited stock, and partner assignment should handle multiple partners contending for the same order. Order placement must atomically reflect item stock, order state, and payment. Status updates should fan out asynchronously to customer, restaurant, and delivery partner without blocking the calling flow. Ratings and reviews after delivery should be supported.

Roles: admin (manage cities, restaurants, and delivery partners), restaurant owner (manage menu and accept or reject orders), customer (browse, order, track, rate), and delivery partner (accept assignments and update status).

Instructions

Framing

The Product Requirement above is intentionally open-ended. You own the scoping decisions — which entities to model, which APIs to expose, which edge cases to handle, and what to leave out. Your interpretation of the requirement and the features you choose to build are themselves part of what is being evaluated. You are encouraged to interpret the requirement generously and submit a feature-rich solution — both the breadth and the depth of the features you build contribute to the evaluation. Document every meaningful assumption in the `README.md` and explain your reasoning in the recorded video.

In Scope

The following are expected in your submission:

- REST APIs covering the core flows
- Persistence to a database of your choice
- Basic role-based access control for the roles defined in the requirement

- Input validation and error handling
- Unit & Integration Tests for the core flows

Out of Scope

Do not spend time on:

- UI or frontend
 - Deployment, containerization, or CI/CD
 - Distributed systems or microservices
 - Advanced authentication (OAuth, SSO, MFA)
 - Production-grade observability, monitoring, or alerting
-

What to Submit

- **GitHub repository** (mandatory)
 - Your personal Github Project repository link
 - Multiple commits are expected during the development phase
 - Must include a `README.md`
 - Must include the `Agents.md` / `Claude.md` file used during development
 - Must include the skills used during development
 - Must include all raw files used during development
- **Video recording** (maximum 10 minutes [loom video](#)) in which you explain:
 - How you approached the problem and the solution at a high level
 - The tech stack used and the reasoning behind it
 - The AI workflow used
 - The testing approach